

시간 제약 kNN 검색과 Multi-Level Block Indexing

Efficient Time-Restricted kNN Search in High-Dimensional Data Using Multi-Level Block Indexing, with Extensions to Multi-Attribute Filtering

Jisoo Kang · Changhun Han · Suji Kim · Ha-Myung Park (2026)

검색 범위가 좁아도 넓어도 대응하는 다중 해상도 인덱스

벡터 유사도와 시간 조건을 동시에 만족해야 한다

가까운 벡터를 찾되, 지정한 시간 구간 밖의 벡터는 결과가 될 수 없다.

01	질의 벡터 w	유사도 기준점
↓		
02	시간 창	허용 후보 제한
↓		
03	거리 계산	후보 안에서 순위
↓		
04	top-k	가장 가까운 k개

필터와 근접 검색 중 어느 것을 먼저 하느냐가 시간 창 크기에 따라 유불리를 바꾼다.

T-kNN 질의의 정확한 의미

$q=(w,k,t_s,t_e)$ 는 벡터, 결과 수, 시작 시각, 종료 시각을 함께 지정한다.

기호	뜻	판정
w	질의 벡터	거리의 기준
k	반환할 이웃 수	최종 결과 개수
t_s	포함되는 시작	$t_s \leq t$
t_e	제외되는 끝	$t < t_e$
$D(t_s:t_e)$	허용된 벡터 집합	반개구간 (t_s,t_e)

종료 시각 t_e 에 정확히 놓인 벡터는 후보에 포함되지 않는다.

이 논문이 푸는 것은 approximate kNN이다

빠른 검색을 위해 graph-based ANN을 사용하고, 정확도는 recall@k로 통제한다.

- 결과는 시간 또는 속성 조건을 만족해야 한다.
- 거리 기준의 진짜 top-k와 완전히 같다는 보장은 없다.
- 실험은 목표 recall@k에서 queries per second를 비교한다.
- 따라서 speedup은 같은 recall 조건 안에서 읽어야 한다.

필터 조건의 준수와 최근접 이웃의 정확성은 서로 다른 평가 축이다.

두 단순 전략은 반대 범위에서 강하다

필터를 먼저 할지, 근접 그래프를 먼저 탐색할지가 핵심 선택이다.

방법	먼저 하는 일	강한 조건	약한 조건
BSBF	시간 구간 찾기	좁은 시간 창	넓은 시간 창
AFBF	속성 전수 필터	통과 후보가 적음	통과 후보가 많음
SF	전체 graph 탐색	넓은 필터	좁은 필터
MBI	블록 크기 선택	범위에 맞춰 전환	추가 저장공간 필요

MBI는 하나의 고정 순서 대신, 질의 범위에 맞는 검색 해상도를 선택한다.

BSBF: 시간으로 자른 뒤 전부 비교한다

01 정렬된 시간축 timestamp 순서



02 binary search $[t_s, t_e)$ 경계



03 brute-force 구간 안 거리 계산



04 top-k 가까운 k개

구간 안 벡터 수 b 가 작으면 빠르지만, b 가 전체 n 에 가까워지면 거의 전수 검색이다.

SF: 근접 graph를 걷고 시간 조건으로 거른다

01 임의 시작점 candidate set 생성



02 가까운 정점 graph 이웃 확장



03 시간 검사 $[t_s, t_e)$ 만 채택



04 k개 확보 근사 이웃 반환

시간 창이 좁으면 방문한 가까운 정점 대부분이 탈락해 탐색 범위가 커질 수 있다.

시간 창이 바뀌면 병목도 바뀐다

같은 데이터와 질의 벡터라도 필터 비율이 계산량의 위치를 바꾼다.

시간 창	BSBF의 일	SF의 낭비	유리한 쪽
매우 좁음	소수만 거리 계산	탈락 정점 다수	BSBF
중간	거리 계산 증가	탈락 감소	교차 구간
매우 넓음	거의 전체 비교	대부분 채택	SF
질의마다 변화	고정 전략 불리	고정 전략 불리	MBI 동기

MBI의 목적은 특정 baseline을 없애는 것이 아니라, 적절한 단위에서 사용하게 만드는 것이다.

MBI의 핵심: 같은 데이터를 여러 크기로 본다

작은 시간 블록부터 전체 기간 블록까지, 각 수준에 검색 가능한 단위를 둔다.

01 leaf blocks 짧은 연속 구간



02 pair merge 인접 구간 합치기



03 upper blocks 더 넓은 구간



04 query match 맞는 크기 선택

다중 해상도가 범위 적응성을 주지만, 벡터와 인덱스를 여러 수준에 중복 저장한다.

블록 B_i는 구간만이 아니라 검색 단위다

구성	내용	역할
D _i	timestamped vectors	블록의 데이터
G _i	graph-based kNN index	큰 블록의 근사 검색
B _i .ts	가장 이른 timestamp	겹침 시작 계산
B _i .te	exclusive upper timestamp	겹침 종료 계산
S _L	leaf 최대 크기	깊이·공간·build 조절

블록 선택은 시간 범위를 고르고, 블록 내부 index는 고차원 근접 탐색을 말한다.

시간 블록은 포함 관계로 계층을 이룬다

아래는 구조를 읽기 위한 예시이며, 모든 구간은 끝점을 제외한다.

수준	블록 구간	포함 관계
root	$[0, 16)$	전체
level 1	$[0, 8)$, $[8, 16)$	root를 둘로 분할
level 2	$[0, 4)$, $[4, 8)$	왼쪽 parent의 children
level 2	$[8, 12)$, $[12, 16)$	오른쪽 parent의 children

인접한 두 child의 데이터 합집합이 parent 데이터를 만든다.

증분 삽입은 가장 최신 leaf에서 시작한다

01 새 벡터 도착 최신 timestamp



02 마지막 leaf 빈 자리 확인



03 vector append D_i 에 추가



04 leaf full? S_L 도달 검사

이 경로는 새 timestamp가 기존 모든 timestamp보다 늦다는 가정에 의존한다.

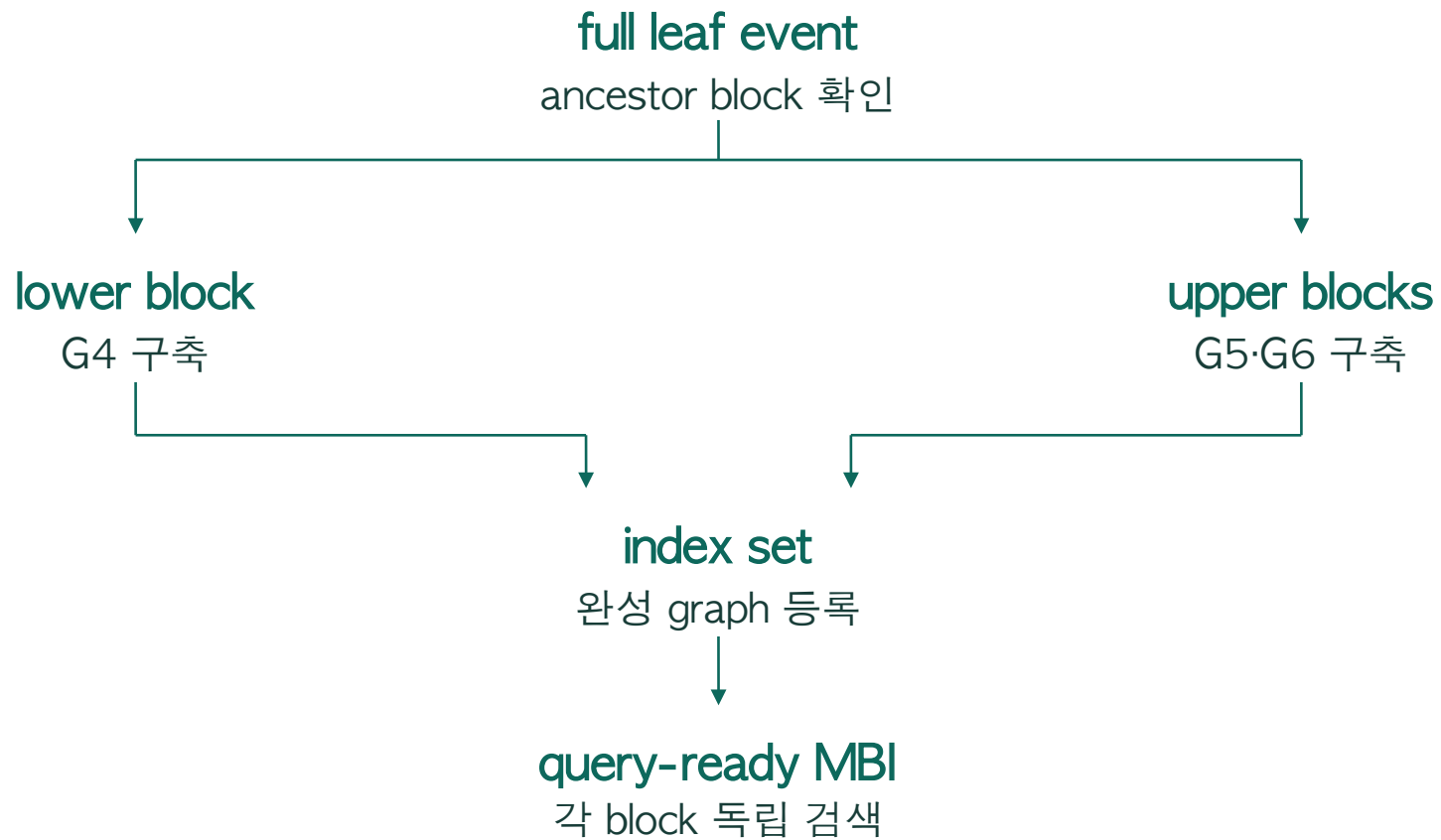
leaf가 차면 아래에서 위로 index를 갱신한다

새 leaf의 graph를 만든 뒤, sibling과 합쳐 필요한 ancestor graph를 구축한다.



부모 graph는 child graph를 이어 붙이는 것이 아니라, 합친 vector set에서 새로 구축한다.

서로 다른 블록 index는 병렬로 만들 수 있다



virtual block은 불완전한 tree의 탐색 경로를 유지한다

- 증분 삽입 중에는 완전 이진 tree가 아닐 수 있다.
- virtual block은 시간 범위를 음의 무한대부터 양의 무한대로 둔다.
- 무한한 block 길이 때문에 threshold 선택 대상이 되지 않는다.
- 대신 실제 descendant까지 top-down 탐색을 이어 준다.

virtual block은 데이터나 결과를 담은 검색 블록이 아니라 구조적 연결 장치다.

질의는 block 선택, 내부 검색, 전역 병합으로 진행된다



선택된 block들은 시간상 겹치지 않으며, 최종 k개는 모든 block 후보를 함께 비교해 정한다.

overlap ratio는 block 중 실제 질의가 차지하는 비율이다

$r_o = \text{겹치는 시간 길이} / \text{block의 전체 시간 길이}$

block	query와 겹침	block 길이	r_o
[0,16)	[5,14): 길이 9	16	$9/16=.5625$
[0,8)	[5,8): 길이 3	8	$3/8=.375$
[4,8)	[5,8): 길이 3	4	$3/4=.75$
[8,16)	[8,14): 길이 6	8	$6/8=.75$

같은 질의라도 작은 block에서는 overlap ratio가 커질 수 있다.

top-down block 선택의 세 경우

조건	동작	이유
$r_o = 0$	skip	질의와 겹치지 않음
leaf	select	더 작은 실제 block이 없음
r_o 가 threshold 충족	select	현재 해상도가 충분함
internal, threshold 미충족	descend	더 맞는 child 탐색

tau는 검색 정확도의 임계값이 아니라 block 해상도를 고르는 임계값이다.

tau가 block 해상도를 조절한다

tau	선택 경향	장점	위험
0에 가까움	root 쪽 큰 block	block 수 감소	좁은 창에서 SF 낭비
약 0.5	중간 해상도	실험상 안정적	dataset tuning 필요
1에 가까움	leaf 쪽 작은 block	불필요 후보 감소	block 수와 overhead 증가

tau를 높이면 항상 빨라지는 것이 아니라, 검색 block 수와 내부 낭비 사이의 균형이 바뀐다.

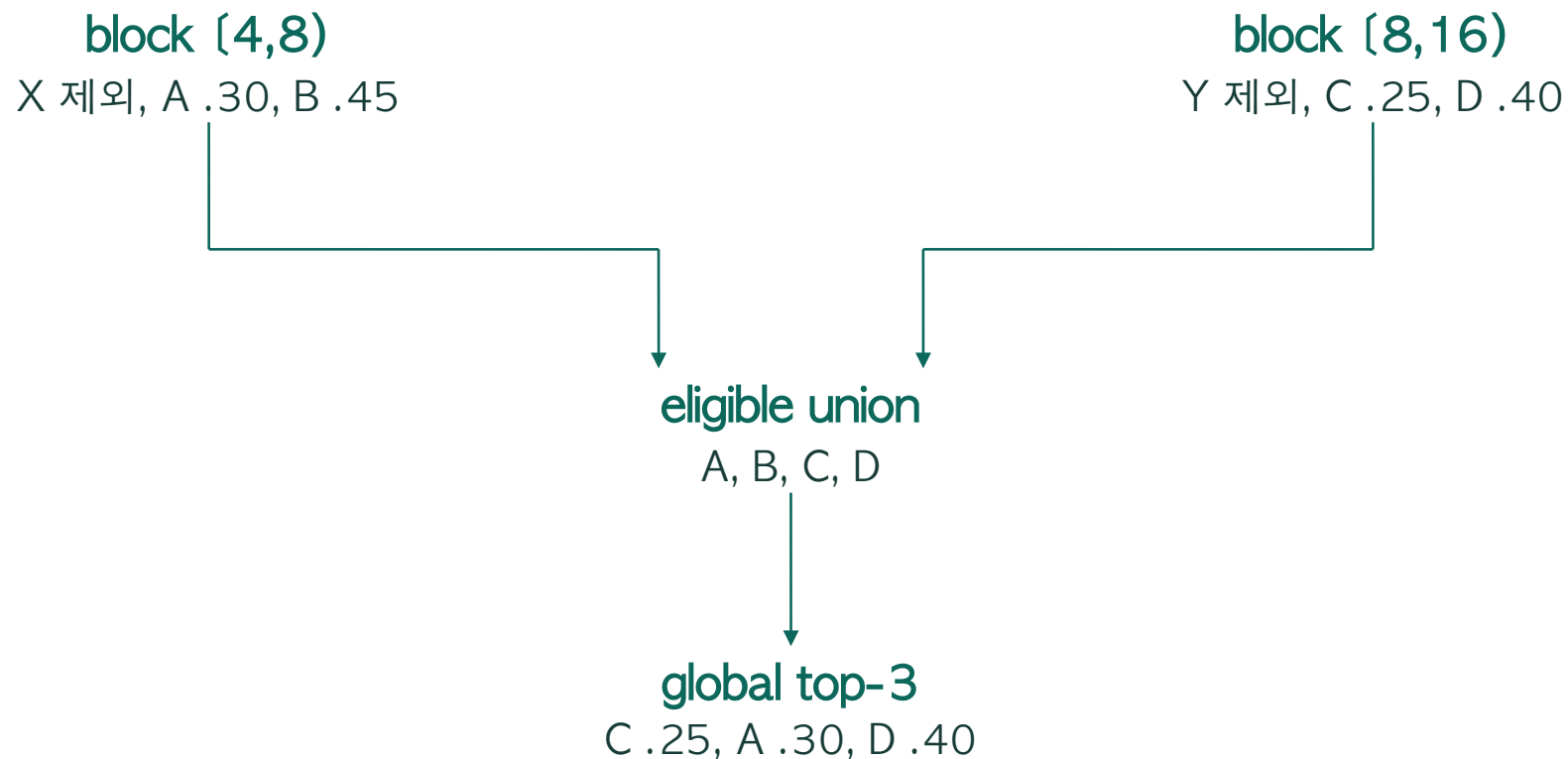
구성 예제: $q=[5,14)$, $\tau=.6$ 에서 block 고르기

논문 규칙을 작은 균형 tree에 적용한 교육용 계산이다.

검사 block	r_o	판정	다음
$[0,16)$	.5625	미충족	두 child로 이동
$[0,8)$	.375	미충족	두 child로 이동
$[0,4)$	0	skip	종료
$[4,8)$	.75	select	검색 집합에 추가
$[8,16)$	.75	select	검색 집합에 추가

최종 search block set은 $\{[4,8), [8,16)\}$ 이고 두 block은 시간상 겹치지 않는다.

구성 예제: block 탐색 결과를 전역 병합한다



선택 뒤에는 block 상태에 따라 내부 검색이 달라진다

선택 block	내부 index	사용 방법	후처리
non-full leaf	비어 있음	BSBF	시간 조건 유지
full leaf	graph 있음	SF	시간 조건 검사
internal block	graph 있음	SF	시간 조건 검사
여러 block	각자 독립	병렬 가능	union 후 global top-k

MBI는 block selection만 제안하는 것이 아니라, block 상태와 기존 검색법을 결합한다.

$\tau \leq .5$ 이면 처리 block은 최대 두 개다

- 질의 창을 처음 둘로 나누는 parent를 기준으로 본다.
- 왼쪽 조각은 한 descendant 경로에서만 선택된다.
- 오른쪽 조각도 대칭적으로 한 경로에서만 선택된다.
- 따라서 각 조각당 한 block, 전체 최대 두 block이다.

작은 τ 영역의 block 호출 수는 tree 깊이에 비례해 폭증하지 않는다.

이론 비용은 index 선택과 graph 가정에 함께 의존한다

대상	논문 결과	전제
index size	$O(D \log D)$	수준별 중복
total build	$O(D ^{1.14})$	NNDescent 경험 모델
amortized append	$O(D ^{0.14})$	ordered arrival
query, $\tau \leq .5$	$O(\log(N_q/\tau) + k/\tau)$	최대 두 block
query, $\tau > .5$	$O(\log^2(N_q/\tau) + (k/\tau)\log(N_q/\tau))$	여러 level 가능

N_q 는 query 시간 창 안 벡터 수이며, 상한은 분석 가정에 조건부다.

빠른 범위 적응의 대가는 level별 중복이다

01 leaf copy 짧은 구간 vector



02 parent copy child union 저장



03 ancestor copies 여러 level 반복



04 total space $O(|D| \log |D|)$

MBI는 한 벡터를 한 번만 저장하는 index가 아니며, 검색 속도와 공간을 교환한다.

m-AkNN은 시간 하나를 m개 수치 속성으로 확장한다

요소	T-kNN	m-AkNN
filter axes	timestamp 1개	numeric attributes m개
query bounds	$[t_s, t_e)$	각 축 $[lower_i, upper_i)$
tree	binary	2^m -ary
non-full leaf	BSBF	AFBF
updates	ordered append	static only

이 확장은 index 구성과 update 가능성을 함께 바꾼다.

multi-attribute 공간은 밀도에 따라 나눈다

01 root range 속성별 min·max



02 midpoint split 각 축을 둘로



03 2^m children vector 재배치



04 size $\leq S_L$ leaf에서 종료

m=2는 quadtree, m=3은 octree이며 데이터가 밀집한 영역이 더 깊게 나뉜다.

여러 속성의 overlap은 축별 비율을 곱한다

아래 수치는 공식을 읽기 위한 구성 예제이며, 축별 비율을 product로 결합한다.

속성	block 범위	query 범위	겹침 비율
a1	[0,10)	[2,8)	$6/10=.6$
a2	[0,20)	[5,15)	$10/20=.5$
product	두 축 결합	동시 조건	$.6 \times .5=.3$

한 축에서 overlap이 0이면 product도 0이므로 해당 block 전체를 건너뛴다.

m-AkNN query는 같은 선택 틀에 다른 filter를 넣는다

01 m ranges 속성별 반개구간



02 product overlap top-down 선택



03 AFBF 또는 SF block 내부 검색



04 global top-k 후보 병합

시간축의 binary search를 쓸 수 없으므로 non-full leaf에서 AFBF를 사용한다.

실험 결과는 이 구현과 장비 조건에서 측정되었다

항목	조건	읽는 법
CPU	AMD Ryzen 7 3800XT	8-core commodity CPU
memory	128 GB DDR4	실험 장비의 용량
OS	Ubuntu 20.04.1	동일 환경 비교
implementation	Rust/Cargo 1.66.1 release	최적화 build
graph index	NNDescent	MBI와 SF에 공통

보고된 QPS와 build time은 하드웨어·언어·index 구현에 조건부다.

T-kNN은 규모와 차원이 다른 여섯 dataset에서 평가한다

dataset	train	dim	distance
MovieLens	57,571	32	angular
COMS	291,180	128	angular
GloVe-100	1,183,514	100	angular
SIFT1M / GIST1M	1M / 1M	128 / 960	Euclidean
DEEP1B subset	9,990,000	96	angular

DEEP1B라는 이름과 달리 실험 train set은 약 999만 벡터의 subset이다.

m-AkNN dataset에는 원문 내부 차원 표기 차이가 있다

dataset	train	Table 2	본문 설명
COMS-GTS	25,544	128-d	temperature·humidity 추가
CORD-19	49,800	728-d	768-d SPECTER라고 서술
query	각 200 제외	held-out	index에 미포함

CORD-19은 표의 728과 본문의 768이 충돌하므로 하나를 사실로 단정하지 않는다.

T-kNN 성능 비교의 operational definition

측	설정	의미
queries	dataset별 200	index에서 제외
window ratio	1%–95%	전체 D 대비 eligible fraction
accuracy	recall@k=.995	같은 정확도 수준 비교
k	10, 50, 100	기본값 10
repeats	각 과정 5회	QPS 반복 측정

10.88×는 recall, window ratio, k, tuning을 고정한 실험점에서의 최대값이다.

MBI는 시간 창 길이에 걸쳐 교차점을 완화한다

관찰	근거	경계
BSBF 하락	window가 넓을수록 QPS 감소	eligible count 증가
SF 하락	window가 좁을수록 QPS 감소	filter reject 증가
MBI 안정	여섯 dataset 전 범위 우세	tuned tau 사용
최대 10.88×	faster-of-BSBF/SF보다 빠름	hypothetical baseline

최대 speedup은 모든 질의의 평균이나 보장값이 아니라 Figure 7에서 관찰된 조건부 최대다.

recall-speed curve 전체를 봐야 한다

- epsilon을 1.0부터 1.4까지 0.02 간격으로 바꾼다.
- 각 방법의 recall과 QPS trade-off를 곡선으로 만든다.
- Figure 8은 COMS의 10%, 30%, 80% window를 비교한다.
- recall=.995의 한 점뿐 아니라 frontier 형태도 확인한다.

정확도를 낮춰 얻은 속도를 동일 정확도의 개선으로 읽으면 안 된다.

SIFT 1M scaling은 관찰된 slope로 보고된다

측정	MBI	SF	해석
indexing slope	1.29	1.13	log-log fitted trend
index-size slope	1.29	1.06	MBI 중복 반영
data 2×	time 약 2.45×	—	$2^{1.29}$ 계산
parallel build	최대 5.08× 감소	—	block 독립성 활용

이 수치는 SIFT 1M subset의 관찰값이며 보편적 계수가 아니다.

실제 index 크기도 MBI의 공간 대가를 보여 준다

dataset	input GB	MBI	SF
MovieLens	.01	.06 (6.08×)	.02 (1.90×)
COMS	.15	.91 (6.35×)	.27 (1.74×)
GloVe-100	.48	3.95 (8.72×)	1.21 (2.49×)
GIST1M	3.85	7.73 (2.15×)	4.64 (1.21×)
DEEP1B subset	3.92	18.24 (5.00×)	6.10 (1.56×)

속도 개선과 함께 index 용량 예산도 실제 배치를 제한할 수 있다.

S_L과 tau는 서로 다른 비용을 조절한다

parameter

작게 두면

크게 두면

논문 관찰

S_L

tree 깊고 index 많음

leaf scan 커짐

depth 5-10 균형

tau

큰 block, 적은 호출

작은 block, 많은 호출

약 .5가 대체로 양호

epsilon

좁은 graph 탐색

넓은 graph 탐색

recall-QPS 조절

tau는 block 선택, epsilon은 graph 탐색 범위를 조절한다.

m-AkNN 결과도 별도의 정확도와 반복 조건을 가진다

측	조건	결과 해석
queries	dataset별 200	index에서 제외
filter ratios	10%, 30%, 80%	qualifying vector 비율
accuracy	recall@k=.95	T-kNN의 .995와 다름
repeats	3회	T-kNN의 5회와 다름
maximum	최대 $1.99\times$	hypothetical baseline 대비

$1.99\times$ 와 $10.88\times$ 는 서로 다른 문제·recall·반복 조건에서 나온 최대값이다.

m-AkNN에서도 MBI index는 SF보다 크다

dataset	input GB	MBI	SF
COMS-GTS, m=2	.01	.08 (5.66×)	.03 (2.29×)
COMS-GTS, m=3	.01	.07 (4.59×)	.03 (2.30×)
CORD-19-small	1.57	2.11 (1.34×)	1.65 (1.05×)
CORD-19-large	3.05	4.23 (1.38×)	3.21 (1.05×)

index 배율은 속성 수뿐 아니라 분포와 tree 모양에도 좌우된다.

MBI가 보장하거나 지원하지 않는 것

경계	현재 범위	의미
neighbor accuracy	approximate	exact top-k 보장 아님
T-kNN updates	시간순 append	임의 삽입·삭제 미지원
m-AkNN updates	static dataset	삽입·삭제 미지원
space	$O(D \log D)$	level별 중복
optimality	open problem	형식적 최적성 미증명

MBI는 범위 적응형 설계지만 모든 동적 update를 지원하지 않는다.

재현 시 함께 고정해야 할 항목

- dataset split과 query 제외 절차
- distance metric과 vector dimension
- recall@k 목표와 k 값
- NNDescent 및 graph search parameter
- tau와 S_L의 dataset별 설정
- CPU·memory·compiler release 조건

공개 코드는 출발점이지만, 논문 표의 parameter와 평가 조건을 함께 맞춰야 한다.

MBI를 한 문장씩 연결하면

01	문제	filter 폭이 매번 변함
↓		
02	구조	여러 크기 block index
↓		
03	결정	overlap과 tau로 선택
↓		
04	실행	block 검색 후 top-k
↓		
05	대가	중복 공간과 제한된 update

MBI의 본질은 질의 범위에 맞는 검색 단위를 고르는 계층적 조정이다.